

A Conceptual Report on CSV Parsing, Data Cleaning, and Visualization in Python

Software/ML Domain Taskphase — Tasks 1 to 5

Participant Name: Penta Divyanshu

Student ID: 251090051426

Domain: Software / ML

Repository: https://github.com/divyanshu1888/Synergy_TP (branch: main)

Submission Date: June 28, 2026

Abstract

This report presents the theoretical foundation behind Tasks 1 to 5 of the Software/ML domain taskphase, completed inside the *Synergy_TP* repository. Rather than walking through the submitted code, it explains the reasoning behind each stage of the workflow: building a reproducible Python environment under Git version control, applying core Python constructs such as functions, dictionaries, file handling, and exceptions, and assembling a small but complete data pipeline. The pipeline begins with parsing a CSV file both manually and with pandas, continues through systematic cleaning of a deliberately messy dataset, and ends with three labelled Matplotlib visualisations. Emphasis is placed throughout on why a particular tool, cleaning rule, or plot type was chosen, and what it reveals about the data, rather than on line-by-line implementation detail.

Introduction

The Software/ML domain taskphase is structured as a progression: each task builds the tools needed for the next. Task 1 establishes the working environment and version-control habits the rest of the project depends on. Task 2 revisits core Python so that later tasks can be written confidently. Tasks 3 to 5 form a single connected pipeline — a CSV file is parsed, cleaned, validated, and finally visualised. This report does not re-describe the code submitted for these tasks. Instead, it explains the concepts that make the code make sense: why CSV parsing needs care, why cleaning decisions must be justified rather than applied blindly, and why a plot is only useful if it is labelled and reproducible. Table 1 summarises the focus and the main concept taken from each task before the later sections discuss them in more depth.

Task	Focus	Main Concept Learned
Task 1	Git, GitHub, virtual environments, Linux basics	Reproducible environments and version-controlled history
Task 2	Python recap: functions, dicts, file I/O, exceptions, type hints	Core building blocks reused throughout the pipeline
Task 3	Manual CSV parser vs <i>pandas.read_csv()</i>	How raw text becomes structured data; what a library automates
Task 4	Cleaning a messy CSV with pandas	Documented, rule-based cleaning and post-cleaning validation
Task 5	Matplotlib visualisation of cleaned data	Choosing the right chart and making it self-explanatory

Table 1: Summary of Tasks 1–5 and the main concept learned from each.

Development Environment and Version Control

Git and GitHub provide the version-control backbone of the project. Git records every change as a commit — a small, named snapshot of the repository — so that the exact state of the code at submission time can be identified by a single commit hash, as required by the submission instructions. This matters more than it might seem: without commit history, there is no reliable way to prove which version of the code actually produced a given output file. Branches exist so that work can be developed in isolation before being merged into a shared line of history (*main*); this project used a single *main* branch throughout, which is reasonable for a solo, linear taskphase but would become important once multiple people, or multiple experimental approaches, needed to coexist.

Repository hygiene was maintained by giving every task its own folder (*task_1* through *task_5*), each with its own README describing how to run it. This keeps tasks independent and reviewable: a reader can evaluate Task 4 without first needing to understand Task 3. A Python virtual environment (*venv*) isolates the project's dependencies from the system-wide Python installation, so that installing pandas or matplotlib for this project cannot silently break, or be broken by, an unrelated project. *requirements.txt* records exactly which packages — and, in Task 1's case, which exact versions, generated with *pip freeze* — are needed, so a reviewer can recreate the same environment with one command instead of guessing. Finally, *.gitignore* excludes machine-specific or regenerable files, such as the *venv/* folder itself, Python's *__pycache__* bytecode cache, and OS files like *Thumbs.db*, keeping the repository focused on source code rather than local build artefacts.

Python and File Handling Concepts

Several Python constructs recur across Tasks 2 to 5 because they map naturally onto tabular data. A dictionary is a natural fit for one CSV row: each column name becomes a key and each cell becomes a value, mirroring how a row is just a set of named fields, and a list of such dictionaries mirrors the whole table. Functions break the pipeline into named, independently testable steps — *read_csv_manual*, *convert_types*, *calculate_summary*, and so on — instead of one long script; this also made it possible to specify exactly which functions each task required, since a well-named function documents what it does without needing comments.

File I/O throughout the tasks uses the *with open(...) as f:* pattern. This context-manager form guarantees the file is closed even if an error occurs while reading it, which matters once exception handling enters the picture: several functions catch *ValueError* or *KeyError* around

type conversion — for example, when a score field is not a valid integer — and skip the offending row with a warning instead of letting the whole program crash. This is a deliberate trade-off, favouring a program that completes with a clearly logged skip over one that halts entirely on the first bad row. Type hints such as *file_path: str* and *-> list[dict]* do not change how the code runs; Python does not enforce them at runtime. Their value is documentation and tooling: a reader, or an IDE, can see what a function expects and returns without reading its body.

CSV Parsing and Pandas

A CSV file is plain text: a header line naming the columns, followed by one line per record, with fields separated by commas. Every value is, at the level of the raw text, a string — there is no built-in distinction between the number 8 and the text "8" until something decides to convert it. Task 3 makes this explicit by building a parser by hand: it reads the file line by line, splits the header and each row on commas, and pairs them into a dictionary, skipping blank lines and any row whose number of fields does not match the header. Only after this structural parsing does a separate step, *convert_types*, decide that the score column should become an integer and that the submitted column should become a boolean.

pandas' *read_csv()* performs the same structural parsing and type inference automatically, and replaces explicit loops with vectorised operations such as *groupby* and *mean*. Building the manual version first, and then reproducing the same nine summary statistics with pandas, served as a way to check that the manual implementation was not just superficially working but actually correct: in this project, every statistic — totals, averages, the highest and lowest scorers, and the per-domain averages — matched exactly between the two implementations. This is also the practical argument for preferring a library like pandas in real work: the manual parser is valuable for understanding what is happening underneath, but it does not handle quoted commas, multi-line fields, or large files efficiently, all of which pandas handles by default.

Data Cleaning

Task 4's input file was deliberately messy in several independent ways, and each needed its own rule rather than a single blanket fix. Categorical values were inconsistent ("ml", "ML", and "MACHINE LEARNING" all mean the same domain) and were mapped to one canonical label per domain. Numeric fields were recorded in mixed units or as words instead of digits —

heights as "170 cm" or "1.62 m", weights as "65 kg" or "55kg", scores and study hours occasionally spelled out ("nine", "two") — so each column needed its own conversion function before it could be treated as a number at all. One row (student S005) was an exact duplicate and was dropped outright, since keeping it would silently double-count that student in every later average.

Two further situations needed an explicit, documented choice rather than a default. First, some attendance values fell outside the only sensible range for a percentage — a recorded -10% and a recorded 105% — and were treated as invalid and replaced, since deleting the row would also discard that student's otherwise-valid score and study-hours data. Second, several cells were genuinely missing. Both the invalid attendance values and the missing numeric values were filled using each column's median rather than its mean, because a median is far less sensitive to the small number of extreme or invalid values already present in this dataset, and because filling rather than dropping preserved every student's other recorded fields. Table 2 records each issue alongside the rule applied to it.

Data Issue	Example	Cleaning Rule Applied
Inconsistent categories	"ml", "MACHINE LEARNING"	Mapped to one canonical label ("ML")
Mixed height units	"170 cm", "1.62 m"	Converted everything to centimetres
Mixed weight units	"65 kg", "55kg"	Converted everything to kilograms
Numbers written as words	"nine", "two"	Mapped word $\rightarrow$ digit before numeric conversion
Out-of-range percentage	-10% , 105%	Treated as invalid; replaced with column median
Missing values	Blank attendance / score cells	Filled with the column median
Duplicate record	Student S005 repeated	Exact duplicate row dropped

Table 2 (optional): Data issues identified in Task 4 and the rule used to resolve each one.

Cleaning alone is not sufficient: `validate_cleaned_data()` exists because a cleaning script can have bugs of its own, and the only way to be confident the output is actually usable is to check it against explicit rules after the fact — no duplicate IDs, attendance within $0-100$, every numeric column actually numeric, domain and submission values restricted to their allowed sets, and no missing values left in any column the rest of the pipeline depends on. Without this step, a silent

cleaning error could propagate into the summaries and plots in Task 5 without ever being noticed.

Visualization

Figure 1 places Task 5 in the context of the whole pipeline: the cleaned, validated CSV from Task 4 is the only input, and three PNG plots are the output.

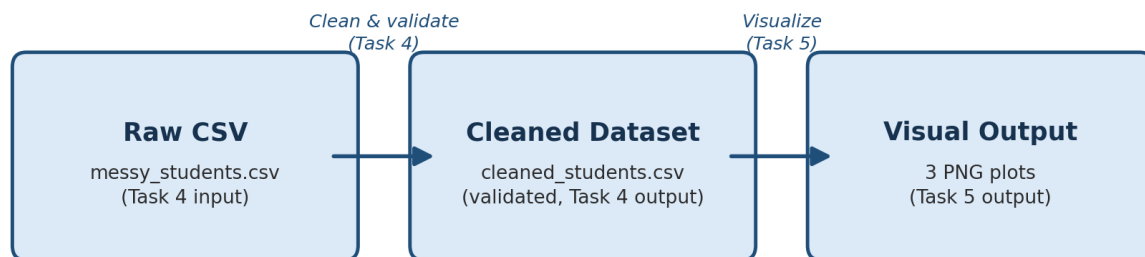


Figure 1: Workflow from raw CSV data, through cleaning and validation, to the final plots.

Each plot in Task 5 was chosen to answer a different kind of question. The average-score-by-domain bar chart compares a single summary number across categories, and is the right tool when the question is "which group did best overall" rather than "how is any individual student doing." In this dataset it also exposes a limitation worth noting in any report, not just this one: the Mechanical bar is exactly zero because only one Mechanical student exists in the cleaned data, so that "average" is really a single data point and should not be read as a stable group trend. The attendance-versus-score scatter plot exists to ask a different question — whether two continuous variables move together — by showing every student as one point rather than collapsing them into an average; here the points are spread fairly broadly across the attendance range with no obviously strong trend, which is itself a useful, if negative, finding. The submission-status count is a simple bar chart of a single categorical variable, included because "how many submitted" is a basic health check on the dataset that does not require any numeric comparison at all.

Across all three, a title and labelled axes are not decorative — a saved PNG has no code attached to it, so anyone looking at the image later has only the title and axis labels to know what it shows. Saving each figure with `savefig()` rather than displaying it in an interactive window also makes the plotting step scriptable and repeatable: the same three files are produced every time `main.py` is run, with no manual step in between. `tight_layout()` was used to stop longer axis labels, such as the rotated domain names, from being clipped at the edge of the saved image — a small detail, but one that affects whether the figure is actually readable once removed from the terminal it was created in.

Conclusion

Taken together, Tasks 1 to 5 form one continuous story: a raw CSV file becomes a cleaned, validated dataset, and that dataset becomes three readable figures. The earlier tasks set up the conditions for this to be possible at all — a controlled environment, a version-controlled history, and a working knowledge of the Python constructs the later tasks rely on. The main technical lesson from Tasks 3 to 5 is that none of the individual steps — parsing, cleaning, plotting — are difficult in isolation, but each one requires a deliberate decision: how to treat a malformed row, which value to impute and why, which chart answers which question. It is the documentation of those decisions, not the code itself, that turns a script into something another person can trust.

References

1. Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
2. pandas Development Team. pandas Documentation. <https://pandas.pydata.org/docs/>
3. Matplotlib Development Team. Matplotlib Documentation. <https://matplotlib.org/stable/>
4. Chacon, S. and Straub, B. Pro Git. <https://git-scm.com/book>
5. Python Enhancement Proposals. PEP 484 – Type Hints. <https://peps.python.org/pep-0484/>